

Point-by-point response to reviewers

Manuscript JRESS-D-26-03751. Reviewer comments are quoted verbatim from the decision letter of 6 July 2026. Every section, table, and figure number below is the current, compiled numbering of the revised manuscript (verified against the source directly, not carried over from earlier drafts). Numbers not spelled out in the manuscript body but needed to answer a comment precisely are given here, in the response, per the manuscript’s own convention of never referencing revision history in its body.

Comments to the Authors (Reviewer #1)

1. Limited benchmark validation

The validation against dPrPm is based on a single 16-node benchmark network. While IPA reproduces the exact solution, the evidence is insufficient to demonstrate superiority or general applicability. Additional benchmark networks with varying sizes and topological complexities should be included.

We agree, and have substantially expanded the validation. Section 5.2 now validates IPA against an independent exact method — a reduced-ordered binary decision diagram (ROBDD) built with CUDD under dynamic variable reordering — across four groups of networks: a corpus of 129 random and mutated DAGs of 10 to 28 nodes and densities 0.14–0.42 (Table 6); six structured topological families (multi-source, grid/lattice, layered, bridge, series-parallel, complete); larger random networks of 30 to 50 nodes; and five named structured and real-infrastructure networks — the benchmark grid, a power distribution network, a gas distribution network, a metropolitan transit network, and the EPANET Net3 water distribution benchmark (Table 5). Breadth of topology and scale was additionally tested on 18 published Bayesian-network benchmark topologies of 5 to 1,041 nodes (Section 5.2). Every network was evaluated by both methods; the worst observed disagreement anywhere is floating-point rounding, at most 4.4×10^{-16} .

2. Runtime comparison is not fully convincing

The reported 134× speedup over dPrPm is based on runtimes extracted from a published paper rather than a controlled implementation environment. The manuscript itself acknowledges this limitation. Consequently, claims regarding computational superiority should be moderated or supported through a more rigorous comparison.

We accept this fully. The 134-fold speedup claim has been removed from the manuscript entirely — it does not appear anywhere in the revision, including the abstract and conclusions. The dPrPm runtime is retained in Table 4 only as a quoted, explicitly indicative reference point, with the caption and surrounding text stating plainly that it was obtained in a different implementation environment and that the dPrPm implementation is not publicly available to reproduce. Every quantitative performance claim now made in the manuscript instead rests on a controlled comparison against the ROBDD oracle, run in the same process and on the same machine as IPA (Section 5, opening paragraph), applied consistently across the grid benchmark

(Table 4), the full corpus (Section 5.2), the interval-propagation comparison (Table 7), and the applied case study (Section 5.4.2).

3. Lack of comparison with established exact methods

The manuscript discusses BDDs, cut-set methods, and decomposition methods in the literature review, but no quantitative comparison is provided. Since IPA is presented as an exact reliability algorithm, comparisons with established exact approaches would substantially strengthen the contribution.

A quantitative comparison has been added throughout Section 5.2. For every network in the corpus, we report the maximum conditioning-set size $|C|_{\max}$ IPA encounters, its realised number of distinct conditional sub-problems, and the ROBDD node count under dynamic reordering. Figure 7 shows that $|C|_{\max}$ tracks the logarithm of the ROBDD size across the corpus, confirming that the two methods are governed by the same structural width. The finding is stated honestly: neither method dominates — on some networks IPA resolves fewer sub-problems than the diagram has nodes, on others more — and IPA’s distinctive value is not a smaller exact state space for point-valued computation but the native imprecise-propagation capability of Section 5.3, which the decision diagram does not provide.

4. Theoretical complexity analysis requires clarification

The complexity reduction claimed in Sections 4.3 and 6 appears largely qualitative. The manuscript would benefit from a more rigorous theoretical analysis, including clearer derivation of the proposed complexity expressions and discussion of worst-case behavior.

Section 4.3 has been rewritten around an exact per-instance cost expression, $W = \sum_{d=1}^M 2^{|C_d|} \cdot O(|E_d|)$, computable before enumeration begins because identification produces every conditioning set in advance (this is stated explicitly as a sound a-priori upper bound, not the realised cost, which memoisation reduces further). The section adds a formal positioning of IPA as a specialisation of cutset conditioning, bounding its cost by the same treewidth parameter that governs junction-tree inference and well-ordered decision diagrams, and an explicit worst-case discussion: IPA is exponential in this width, as is every exact method, reflecting the #P-hardness of network reliability. Two new lemmas support this — separator sufficiency (Lemma 4.2) and independent-substructure factorisation (Lemma 4.3) — each with a full proof, and a new adversarial-structure analysis (Section 4.3, “Adversarial structure” paragraph, and Section 5.5) measures the worst-case behaviour directly on constructed and real published network families.

5. Drone logistics case study lacks reliability insights

The second case study focuses almost exclusively on computational performance. However, the paper is framed as a reliability analysis contribution. The manuscript should present actual reliability results, engineering interpretations, or decision-support implications obtained from the drone network analyses, not only runtime statistics.

The case study has been rebuilt entirely (Section 5.4) around a real, cited source design study — a medical drone logistics network for Scotland — and now reports engineering reliability findings directly (Section 5.4.3, “Reliability findings”). In the two higher-redundancy configurations, the best-connected facilities are reachable with near-certainty while the worst-served facility, an island hospital, has reachability bounded between 0.56 and 0.72: a decision-relevant spread no point-probability analysis of the same design could distinguish from a confident single value of 0.6, yet the two call for different engineering interventions. A reliability map (Figure 10)

locates where this uncertainty concentrates, and the redundancy/tractability trade-off is itself presented as a decision-relevant finding: provisioned redundancy improves resilience and raises the cost of verifying it exactly, measured on the real network rather than asserted qualitatively.

6. Scalability limitations deserve deeper discussion

The results indicate that networks with nesting depth 16-18 become computationally expensive or impractical. Since this limitation is central to the applicability of the method, the manuscript should discuss more explicitly the practical range of network sizes and structures for which IPA remains competitive.

We agree this deserves deeper, more precise discussion, and Section 5.4.2 and the “Practical range” paragraph of Section 5.6 now do so with a measured quantity, not the qualitative “nesting depth” framing of the original submission. The original six-Pareto case study reported computation time as a function of a *nesting depth* read off the constructed network’s own layering; the revised case study reports the same underlying phenomenon in terms of the method’s own cost parameter, the maximum conditioning-set size $|C|_{\max}$, measured directly rather than approximated by a proxy. On the real network now used, a conditioning requirement of 16 — reached at sixteen provisioned alternate routes per location, a genuine network-design choice — remains practical (full exact interval propagation completes in under 25 seconds), while connecting every operationally plausible pair of locations makes exact diamond identification itself exceed a 16 GB memory budget before propagation can even begin (Section 5.4.2). This is the same qualitative boundary the original submission located near nesting depth 16–18; the revision states it as a measured conditioning-set size on a real infrastructure network instead of an approximated depth on a constructed one, and Section 5.6 additionally proposes a depth-limited hybrid (condition exactly to a chosen width, bound rigorously beyond it) for networks past this boundary, explicitly labelled as proposed and not yet implemented.

7. Algorithm reproducibility

Algorithm 1 provides only a high-level workflow. Additional implementation details regarding diamond identification, conditioning-set determination, and supernode caching would improve reproducibility.

A second, fully specified algorithm (Algorithm 2, Section 4.2.2) now gives the diamond identification procedure explicitly: it partitions a join’s parents into independent ancestry groups by testing pairwise intersection of unresolved influence (Lemma 4.3), finds each group’s covering fork, and recurses into nested diamonds under an extended conditioning context. Section 4.2.2 additionally specifies exactly how a resolved diamond is keyed for reuse — by its edge list together with the outer conditioning context that fixes it (Equation 21), not by its edge list alone — which is precisely what lets the same substructure be identified as a distinct sub-problem under a different outer context and cached correctly rather than incorrectly reused. Section 4.4 specifies the two caches the implementation maintains and what each avoids recomputing. A traced, four-diamond worked example (Section 4.2.1, Table 1) follows the whole procedure step by step on a nested network, beyond the single-diamond example of Section 3.3.

8. Minor editorial issues

Several grammatical and stylistic issues are present throughout the manuscript. Examples include duplicated punctuation, typographical errors, and lengthy sentences in Sections 1, 4, and 5. A thorough language revision is recommended.

A language pass has been completed. The manuscript was read in full for duplicated punctuation, typographical errors, and unnecessarily long or run-on sentences in the sections named,

and elsewhere; a number of genuine issues were found and corrected, including dead/unused notation, a subject–verb agreement error, an internally inconsistent claim about BDD space complexity, a section miscount in the Section 5 introduction, and redundant passages restating the same definition or claim twice. The sections newly written for this revision (5.2 through 5.6) were drafted to the same standard from the outset.

Reviewer #2

1. Parallels with Cutset Conditioning / Junction Tree — justify novelty

The core mechanism of IPA (resolving dependencies via conditioning and supernode caching) strongly parallels established exact inference techniques in probabilistic graphical models, such as Cutset Conditioning and the Junction Tree Algorithm. To ensure theoretical novelty, the manuscript must compare IPA to these existing algorithms and mathematically justify how it improves and specializes the theory specifically for graph-based process systems.

Section 4.3 (“Relation to cutset conditioning and junction trees”) now makes this relationship formal and states it candidly rather than downplaying it. IPA is a specialisation of cutset conditioning to source-to-node reachability: rather than selecting one global cutset whose enumeration renders the whole network singly connected (cost exponential in the cutset size), IPA identifies a local separating set at each reconvergence it actually visits (Lemma 4.2), conditions on it, and recurses, so the realised cost is a sum of local exponentials rather than one global exponential. A new factorisation lemma (Lemma 4.3) further reduces fan-in cost from exponential to linear where forks are independent given the current context. IPA’s cost is governed by the same treewidth-bounded parameter as junction-tree inference. We do not claim asymptotic superiority; the claimed contribution is the specialisation itself — local conditioning, factorisation, and context-keyed reuse — together with the capability it enables and general PGM inference engines do not provide: native exact propagation of interval- and probability-box-valued inputs (Section 5.3).

2. Benchmark against state-of-the-art exact solvers on the same grid and drone networks

In Section 5.1, the proposed exact method is benchmarked solely against the directed Probability Propagation Method (dPrPm), which is an approximate bounding technique. From the reviewer’s perspective, IPA must also be benchmarked against state-of-the-art exact solvers, such as optimized Binary Decision Diagrams (BDDs) or standard exact inference engines, using the same grid and drone networks.

Done, on both networks named. On the grid benchmark, IPA and a sifted ROBDD agree at every node to 1.1×10^{-16} (Table 3), and Table 4 reports each method’s own build-and-evaluate cost on the same machine. On the drone case study (Section 5.4.2), the same comparison is repeated on all three network configurations: on the centralised configuration the two methods agree to floating-point precision and IPA’s one-shot interval computation is about 56 times faster than the ROBDD route (0.008s against 0.463s); on the minimal-investment topology at six alternate routes per location, agreement is 1.1×10^{-16} and IPA is about 15 times faster; at sixteen alternate routes per location on the same topology, the ROBDD build did not complete within a practical budget under either of two variable-ordering strategies, while IPA completed the full exact computation in under 25 seconds. The claim is scoped precisely: this is a measured result on this network, not a general claim that decision-diagram methods cannot handle this problem class.

3. DAG transformation of the bidirectional multiplex network

Section 5.2.1 forces the bidirectional multiplex logistics network into a directed acyclic graph (DAG) by restricting edges to flow only from lower to higher functional levels. Because realistic modeling and decision-making in critical infrastructures often rely heavily on cyclic dependencies and bidirectional flows (e.g., returning drones or feedback loops), the authors must address the limitations of this DAG transformation and justify why this topological simplification does not compromise the validity of the resilience assessment.

The case study has been rebuilt, and its directionality is now grounded in the reliability question being asked, not in a modelling convenience (Section 5.4.1). The source study’s own connections are undirected — a route can be flown either way — but the question IPA answers is whether supply reaches each facility from the hubs, which is inherently directional regardless of whether the underlying transport link is bidirectional. The dependency graph is therefore directed from the hub tier outward, matching the source study’s own hub-and-spoke architecture, and this directional question does not require modelling return flights or feedback loops to be answered validly. We also state the limitation explicitly rather than leaving it implicit (Section 5.6, “Structural scope”): general cyclic and bidirectional dependency structures — feedback loops, return flows — are outside the present scope, and their principled treatment (for example by unrolling or by strongly-connected-component condensation) is identified as future work.

4. Scalability limit; strategies for high-treewidth graphs including a hybrid

The authors note that computation times become intractable when diamond nesting depth reaches 18 or more. Since real-world critical infrastructures are often highly dense and deeply nested, this scalability limit is a major constraint. Section 5.3 should be expanded to discuss strategies for managing high-treewidth graphs, including a hybrid approach that uses approximate message-passing for deeply nested supernodes while maintaining exact inference in simpler regions of the network.

Section 5.6 (“Practical range”) now gives the measured practical boundary on a real network in terms of the controllable redundancy design parameter described in our response to Comment #6 above, and outlines a hybrid along the lines suggested: exact conditioning to a chosen depth, with reconvergent contributions beyond that depth combined without conditioning to give a rigorous bound — a sound but wider outer bound for interval inputs, and a Fréchet-bounded combination for probability-box inputs. We are transparent that this hybrid is proposed and analysed qualitatively, not implemented or evaluated in this revision: the method as reported never trades exactness for tractability, and we judged it more honest to state the boundary precisely than to claim an approximation capability we have not validated.

Reviewer #3

1. Detailed pseudocode / multi-level nested example

Section 3 and 4 need significant expansion and reorganization. The workflow (Algorithm 1) is high-level; readers need a more detailed pseudocode or step-by-step example tracing a multi-level nested diamond through decomposition and supernode replacement beyond the simple single-diamond in Section 3.3.

A worked multi-level example has been added (Section 4.2.1): the nested network of Figure 5 is traced step by step through four diamond resolutions (Table 1) — each diamond’s conditioning set, the enumerated conditional signal probabilities, and the belief at each join — showing how inner structures are resolved once and reused by an outer diamond that later re-conditions on

the same fork in a different context. The formal recursion underlying the trace is now fully specified by Algorithm 2 and the four lemmas of Section 4.1.

2. Supernode storage/query; overlapping diamonds; cache explosion

Elaborate on supernode implementation details: How exactly are conditional maps stored and queried? What happens with multiple overlapping diamonds or shared conditioning nodes across diamonds? Address potential cache explosion or state-space management.

Section 4.2.2 specifies the identification of stored sub-problems: a resolved substructure is keyed by the pair of its edge list and its conditioning context (Equation 21), so that the same subgraph reached under a different outer conditioning state is treated as a distinct sub-problem and resolved, and cached, separately — this is precisely the guarantee needed when diamonds overlap or share conditioning variables, and is a direct consequence of the conditional-invariance lemma (Lemma 4.1). Conversely, a conditioning node already fixed by an outer context contributes only one term to the enumeration, not two, so nesting further levels of context does not double the enumeration already performed. Section 4.4 describes the two caches the implementation maintains: one keyed on this identification key, avoiding re-identifying the same diamond, and a second keyed on a diamond’s edge list together with a conditioning state’s node priors, avoiding recomputing a belief when the identical sub-problem recurs under a different diamond or context.

3. Prove Lemmas 4.1/4.2

Please prove or more formally justify Lemma 4.1/4.2 on conditional invariance and supernode equivalence. The current proofs are sketches; strengthen them.

Done. The revision states and proves the conditional-invariance lemma (Lemma 4.1) directly from the component-independence assumption, giving the precise condition under which a resolved diamond’s conditional map is context-invariant (the outer conditioning fixes no variable internal to the resolved substructure), and strengthens the supernode-equivalence proof (Lemma 4.4) accordingly, now building explicitly on the two new lemmas below. Two further lemmas are added and proved: separator sufficiency (Lemma 4.2 — the identified conditioning set restores conditional independence of a join’s parents) and independent-substructure factorisation (Lemma 4.3). A corollary (Corollary 4.5) proves that repeated supernode replacement preserves the acyclicity of the reduced graph, answering part of Comment #4 below with an explicit argument rather than an assertion.

4. Overlapping / non-hierarchical diamonds; no information loss or cycles

In progressive decomposition, clarify handling of overlapping or non-hierarchical diamonds. What guarantees no loss of information or cycles in reduction?

Handled by the context-aware identification of Section 4.2.2 together with the invariance condition of Lemma 4.1: overlap and shared conditioning never cause a stored conditional map to be reused outside its validity condition, because the conditioning context is part of a sub-problem’s identity, not an afterthought. No information is lost, because Lemma 4.4 shows the supernode replacement preserves the exact source-to-node reliability of every downstream node. No cycle can be introduced by reduction: we have added Corollary 4.5, proving explicitly that a supernode’s only edges are its diamond’s own external incoming edges (retained with their original direction) and one outgoing edge to the join, so any cycle in the reduced graph would trace back to a cycle in the original network, which Assumption 2.1 excludes — and the argument is stated for repeated replacement across nested and overlapping diamonds, not just a single one.

5. Complexity: tighter bounds / worst-case discussion

Complexity analysis (Section 4.3) is qualitative. Provide tighter bounds or worst-case discussion, even if empirical results dominate.

Addressed in the rewritten Section 4.3: an exact per-instance cost expression (Equation 22, not an asymptotic guess), a treewidth-bounded width parameter relating IPA to junction trees and well-ordered decision diagrams, an explicit worst-case discussion ($\#P$ -hardness; exponential in width, as for every exact method), and a measured cost comparison across the corpus (Section 5.2, Figure 7) in which neither IPA nor the diagram dominates. Section 5.5 additionally measures worst-case behaviour directly on two constructed adversarial families and a real published circuit benchmark family (ISCAS85), including a genuine finding — a memory-pressure wall at which wall-clock cost stops tracking the realised sub-problem count — reported honestly rather than smoothed over.

6. Drone DAG transparency (probabilities from distances, levels, bidirectionality)

Drone networks are valuable but the DAG construction from multiplex logistics needs more transparency (e.g., how transmission probabilities are derived from distances, exact level assignments, and handling of bidirectional original edges). Provide a small illustrative subgraph or statistics on average in/out-degrees.

The case study has been rebuilt so that every input is traced to the source design study directly (Section 5.4.1): a connection exists between two locations, for a given drone type, exactly when their distance is within that type’s nominal range (matching the source study’s own construction rule); non-hub location availability is represented as a sensitivity interval centred on the source study’s own reported success probability; and direction follows the source study’s own hub-and-spoke supply logic rather than a distance- or level-based heuristic. Exactly one extension is added and flagged as such: connections within the final ten per cent of nominal range are given a vacuous transmission probability, representing unquantified weather derating that the source study itself raises but does not quantify. In place of an illustrative subgraph — at 263 to 1,753 connections per configuration, a raw topology figure would not be legible — Section 5.4.1 now reports degree statistics per configuration: the centralised design is sparse (mean in- and out-degree 1.2, maximum out-degree 27 at a hub) and close to tree-like, while the two denser designs have mean degree 7.2 and individual facilities served by up to 16–17 alternate incoming routes, the same per-facility redundancy that produces the conditioning-set sizes reported in Section 5.4.2.

7. Figure quality and captions

Please ensure all are high-quality, labeled clearly. Add captions explaining key take-aways.

All figures have been reviewed for both content and rendering. Every caption now states a takeaway in addition to describing the figure’s content (for example, Figure 7’s caption states directly that neither method dominates, and Figure 10’s states where uncertainty concentrates and why). The new figures added in this revision — the width-correlation figure (Figure 7), the p-box tightness envelope (Figure 8), the p-box cost-scaling figure, and the drone reliability map (Figure 10) — follow the same convention.

8. Broader applicability (Bayesian networks) or limitations in non-process networks

Discuss broader applicability (e.g., to Bayesian networks or other inference tasks) or limitations in non-process networks.

Addressed explicitly in Section 4.3 (“Applicability beyond source-to-node reachability”) and Section 1.1. We were deliberately careful to scope this claim correctly rather than overstate it: cutset conditioning, the *general* method IPA specialises, already applies to general Bayesian-network inference; what is specific to IPA is a set of closed-form update rules — the independent-union combination of parent signals and the context-keyed supernode cache — derived for the monotone reachability structure of the reliability problem, which do not carry over directly to a Bayesian network with arbitrary conditional probability tables. What transfers is only the width-governed complexity class. This distinction matters more, not less, for the imprecise-propagation capability: for classical, precise Bayesian networks, bounded treewidth guarantees polynomial exact inference, but this guarantee does not survive the move to imprecise, credal parameters, where bounded-treewidth inference remains NP-hard in general (Mauá and Cozman, 2020), and the only known tractable relaxation is an approximation, not an exact one (Fagioli and Zaffalon, 1998). IPA’s width-governed *exact* imprecise result is therefore specific to the arithmetic structure of the reachability problem and the explicit diamond-conditioning construction, not a free consequence of sharing a conditioning strategy with general PGM inference.

Summary of where each concern is addressed

Concern	Manuscript section(s)
Benchmark breadth	5.2, Tables 5–6
Runtime comparison / 134× claim	5, 5.1.2, Table 4
Comparison to exact methods	5.2, Figure 7
Complexity rigour	4.3
Drone reliability insights	5.4.3
Scalability / practical range	5.4.2, 5.6
Algorithm reproducibility	4.2.2, Algorithm 2
Editorial	throughout
Cutset conditioning / junction tree	1.1, 4.3
Same-network BDD benchmark (grid + drone)	5.1.2, 5.4.2
DAG transformation justification	5.4.1, 5.6
High-treewidth hybrid	5.6
Pseudocode / nested example	4.2.1, 4.2.2, Algorithm 2
Supernode storage / cache explosion	4.2.2, 4.4
Lemma proofs	4.1 (Lemmas 4.1–4.4, Corollary 4.5)
Overlapping diamonds / cycles	4.1, 4.2.2, Corollary 4.5
Tighter complexity bounds	4.3, 5.5
Drone DAG transparency / degree stats	5.4.1
Figure quality	throughout
Broader applicability (Bayesian networks)	1.1, 4.3